Guía práctica · Parte 1 de 3

Construyendo el sistema de la heladería, paso a paso

Desde instalar las herramientas hasta tener la app funcionando en tu computadora con React y Supabase. Cada paso dice quién lo hace, dónde y con qué comando, y qué cambiará después en los entornos de pruebas y de producción.

Stack: React + TypeScript + Vite · Supabase (PostgreSQL) · GitHub
Alcance de esta parte: entorno de **desarrollo** (tu computadora). Las partes 2 (pruebas) y 3 (producción) vendrán después.
Noviembre de 2026 · Versiones verificadas: Node 24 LTS, Vite 8, React 19, Supabase CLI 3

Contenido

1. Cómo usar esta guía
2. Lo que vamos a construir
3. Etapa A · Preparar cada computadora
4. Etapa B · GitHub
5. Etapa C · Crear el proyecto React
6. Etapa D · La base del proyecto
7. Etapa E · Supabase en tu computadora
8. Etapa F · Inicio de sesión y roles
9. Etapa G · Los módulos del negocio
10. Etapa H · Funcionar sin internet (PWA)
11. Etapa I · Pruebas automáticas
12. Trabajar desde dos computadoras y con otras personas
13. Lo que viene: pruebas y producción
14. Problemas comunes y soluciones
15. Apéndice A · Comandos del proyecto
16. Apéndice B · Contenido de CLAUDE.md

Leyenda

Etiqueta	Qué significa
TÚ · TERMINAL	Escribes el comando en la terminal de VS Code (*Terminal → New Terminal*, se abre PowerShell) y presionas Enter.
TÚ · NAVEGADOR	Lo haces en una página web (GitHub, Studio de Supabase, la app).
TÚ · VS CODE	Lo haces dentro del editor: abrir carpetas, instalar extensiones, revisar archivos.
CLAUDE	Me lo pides a mí (Claude Code) en VS Code, copiando el texto del recuadro naranja. Yo creo, muevo o borro archivos y escribo el código.

Además verás recuadros verdes ✓ **Comprueba** (qué debes ver para saber que salió bien), azules **En la otra computadora** y morados **Pruebas y producción** (qué cambiará en esos entornos).

1. Cómo usar esta guía

- **Sigue los pasos en orden.** Cada paso supone que el anterior salió bien. Si un "Comprueba" no da lo esperado, no sigas: mira el capítulo 14 o pregúntame pegando el error completo.
- **Tú haces lo que toca tu computadora o la web**: instalar, ejecutar comandos, crear cuentas, configurar páginas. **Yo hago lo que es código**: crear carpetas y archivos, escribir y corregir el código, y explicarte cada cosa.
- **Al pedirme algo, copia el texto del recuadro naranja.** Están escritos para que yo encuentre el contexto aunque abras una conversación nueva o estés en otra computadora: siempre mencionan el paso de esta guía, que estará guardada dentro del proyecto en `docs/`.
- **Después de que yo cree o cambie archivos**, revisa lo que hice (VS Code marca los archivos cambiados en la pestaña *Source Control*) y pregúntame lo que no entiendas. Que el código funcione no basta: tienes que poder explicarlo.
- **Guarda en Git al final de cada etapa.** Así puedes seguir desde la otra computadora y, si algo se rompe, volver atrás.

> **Terminal y PowerShell.** La terminal es una ventana donde se escriben comandos de texto. En Windows, VS Code abre por defecto **PowerShell**. Todos los comandos de esta guía están escritos para PowerShell.

> **Carpeta actual.** Los comandos se ejecutan "dentro" de una carpeta, que aparece a la izquierda del cursor (`PS C:\dev\heladeria-app>`). Si un comando dice que no encuentra `package.json`, casi siempre es porque estás en otra carpeta. Para moverte usa `cd ruta`.

2. Lo que vamos a construir

Un proyecto **nuevo** y separado del prototipo, llamado `heladeria-app`, en un repositorio **privado**. El prototipo (`System-ice`) se queda como referencia: de ahí vamos a llevar la lógica (cálculos de caja, estado de resultados, PDF) y los estilos (los 4 temas).

Las piezas

Pieza	Herramienta	Para qué
Interfaz	React 19 + TypeScript	Pantallas como componentes.
Construcción	Vite 8	Servidor de desarrollo y build de producción.
Navegación	React Router	Pantallas con URL propia: `/` para colaboradores y `/admin` para administración.
Datos	TanStack Query + supabase-js	Leer y guardar en Supabase, con caché y reintentos.
Backend	Supabase (PostgreSQL, Auth, RLS, Edge Functions)	Base de datos, login y permisos.
Sin internet	vite-plugin-pwa + Dexie (IndexedDB)	Instalar como app y anotar sin señal.
Calidad	Oxlint, Prettier, Husky	Revisar y ordenar el código antes de cada commit.
Pruebas	Vitest, Testing Library, pgTAP, Playwright	Comprobar cálculos, pantallas, permisos y flujos completos.

> **Oxlint.** Es el *linter* (revisor de código) que Vite trae ahora por defecto en las plantillas de React. Hace lo mismo que ESLint, pero mucho más rápido.

Los tres entornos, desde ya

Entorno	Base de datos	App	Variables (.env)
Desarrollo esta guía	Supabase local en Docker	`npm run dev` → `localhost:5173`	Archivo `.env.local` en tu PC
Pruebas parte 2	Proyecto Supabase `heladeria-pruebas`	Cloudflare Pages, rama `develop`	Configuradas en Cloudflare y GitHub
Producción parte 3	Proyecto Supabase `heladeria-prod`	Cloudflare Pages, rama `main`	Configuradas en Cloudflare y GitHub

La regla que hace todo fácil después: el código es **el mismo** en los tres entornos. Lo único que cambia son las variables de entorno (a qué Supabase se conecta) y dónde se publica. Por eso, cuando pagues Supabase Pro o cambies de hosting, **no tendrás que cambiar código**: solo valores de configuración.

> **Pruebas y producción:** Lo que hoy pruebas con `npm run dev` y Supabase local será lo mismo que se publicará. Los cambios de tablas viajarán como migraciones (archivos SQL) con `supabase db push`, y la app se publicará sola desde GitHub.

3. Etapa A · Preparar cada computadora

Haz esta etapa **una vez en cada computadora**: la del trabajo y la de tu casa.

> **Antes de empezar en la computadora del trabajo:** instalar WSL2 y Docker requiere permisos de administrador. Además, Docker Desktop es gratis para uso personal y empresas pequeñas, pero **pide licencia pagada en empresas de más de 250 empleados o más de US$ 10 millones de ingresos**. Si la computadora es de una empresa grande, consúltalo o usa **Rancher Desktop** (gratis), que también funciona con Supabase.

TÚ · EXPLORADOR A1. Elegir la carpeta de trabajo

Crea la carpeta `C:\dev`. Ahí vivirán tus proyectos.

¿Por qué no en el Escritorio? En muchas computadoras el Escritorio se sincroniza con **OneDrive**, y eso choca con la carpeta `node_modules` (miles de archivos): se pone lento, aparecen errores raros y llena tu OneDrive. Además, conviene una ruta corta y **sin espacios**.

TÚ · TERMINAL A2. Instalar Git, Node.js y VS Code

Abre **PowerShell** desde el menú Inicio y ejecuta, uno por uno (si ya tienes alguno, winget lo dirá):

```
winget install --id Git.Git -e
winget install --id OpenJS.NodeJS.LTS -e
winget install --id Microsoft.VisualStudioCode -e
```

Cierra y vuelve a abrir PowerShell para que reconozca los programas nuevos.

> **winget.** Es el instalador de programas por comandos que trae Windows. `--id` indica el programa exacto y `-e` pide coincidencia exacta del nombre.

TÚ · TERMINAL (ADMINISTRADOR) A3. Instalar WSL2 y Docker Desktop

Abre PowerShell **como administrador** (clic derecho → *Ejecutar como administrador*):

```
wsl --install
winget install --id Docker.DockerDesktop -e
```

Reinicia la computadora. Después abre **Docker Desktop** desde Inicio, acepta los términos y espera a que abajo a la izquierda diga *Engine running*.

> **WSL2** (*Windows Subsystem for Linux*). Permite correr Linux dentro de Windows. Docker lo usa porque los contenedores de Supabase son de Linux.

Recomendado: en Docker Desktop → *Settings* → *General*, desactiva *Start Docker Desktop when you sign in* si no quieres que consuma memoria siempre. Solo tendrás que abrirlo antes de trabajar.

TÚ · TERMINAL A4. Permitir que PowerShell ejecute los comandos de npm

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Responde `S` (o `Y`) si pregunta. Sin esto, comandos como `npx` fallan con el error "*la ejecución de scripts está deshabilitada en este sistema*".

TÚ · TERMINAL A5. Decirle a Git quién eres

```
git config --global user.name "Juan Jose Huanca Esquivel"
git config --global user.email "juanjosehuanca123@gmail.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
```

Usa el mismo nombre y correo en las dos computadoras: así GitHub te reconoce como la misma persona.

TÚ · TERMINAL A6. Verificar todo

```
git --version
node -v
npm -v
docker --version
docker run --rm hello-world
```

> ✓ **Comprueba:** Git 2.x, Node **v24**.x, npm 11.x, Docker 2x.x. El último comando muestra "*Hello from Docker!*".

TÚ · VS CODE A7. Extensiones de VS Code

En VS Code, panel *Extensions* (Ctrl+Shift+X), instala:

- **Claude Code** (Anthropic), si no la tienes ya.
- **Prettier – Code formatter** (esbenp.prettier-vscode).
- **Oxc** (oxc.oxc-vscode): muestra los avisos de Oxlint mientras escribes.
- **Deno** (denoland.vscode-deno): para las Edge Functions de Supabase.

Más adelante crearé el archivo `.vscode/extensions.json`; en la otra computadora, VS Code te ofrecerá instalarlas todas con un clic.

> **En la otra computadora:** Repite A1 a A7 completos. Es la única etapa que se repite por máquina.

4. Etapa B · GitHub

TÚ · NAVEGADOR **B1. Activar la verificación en dos pasos (2FA)**

github.com → tu foto (arriba a la derecha) → *Settings* → *Password and authentication* → *Enable two-factor authentication*. Usa una app como Google Authenticator o Microsoft Authenticator y **guarda los códigos de recuperación** en un lugar seguro.

Si alguien entra a tu GitHub, entra a todo el proyecto. Después harás lo mismo en Supabase y Cloudflare.

TÚ · NAVEGADOR **B2. Crear el repositorio privado**

1. github.com → botón **New** (o `github.com/new`).
2. *Repository name*: `heladeria-app`.
3. *Description*: *Sistema de ventas y caja para la heladería*.
4. Marca **Private**.
5. **No** marques *Add a README*, *.gitignore* ni *license*. Vite crea esos archivos y, si GitHub también los crea, el primer push choca.
6. **Create repository**. Deja abierta la página: tiene la URL del repositorio, `https://github.com/jj-hesquivel/heladeria-app.git`.

Sobre proteger ramas: en GitHub gratis, las reglas que impiden subir directo a `main` solo funcionan en repositorios **públicos**. En privados requieren GitHub Pro (unos US$ 4/mes). Por ahora lo reemplazamos con disciplina: **nunca trabajes directo en** `main` y usa siempre Pull Requests, como indica esta guía.

Pruebas y producción: GitHub es el centro de todo. Cloudflare Pages leerá este repositorio para publicar `develop` (pruebas) y `main` (producción), y GitHub Actions aplicará las migraciones a cada Supabase.

5. Etapa C · Crear el proyecto React

TÚ · TERMINAL **C1. Generar el proyecto con Vite**

```
cd C:\dev
npm create vite@latest heladeria-app -- --template react-ts
```

Si hace preguntas: si ofrece instalar y arrancar de inmediato (*install and start now*), responde **No**. Si pregunta qué linter usar, elige **Oxlint** (es el recomendado).

> **npm create vite@latest.** Descarga la versión más nueva del generador de proyectos de Vite y lo ejecuta. `--template react-ts` elige la plantilla de React con TypeScript. Los dos guiones `--` separan las opciones de npm de las del generador.

TÚ · TERMINAL **C2. Instalar y probar que arranca**

```
cd heladeria-app
npm install
npm run dev
```

> **✓ Comprueba:** La terminal muestra `Local: http://localhost:5173/` . Ábrelo con Ctrl+clic: aparece la página de ejemplo de Vite + React con un contador.

Para detener el servidor: en la terminal, **Ctrl+C**.

> **npm install.** Lee `package.json` y descarga las dependencias a la carpeta `node_modules` . Esa carpeta **nunca** se sube a Git: cada computadora la genera con `npm install` . El archivo `package-lock.json` sí se sube, porque anota las versiones exactas para que las dos computadoras instalen lo mismo.

TÚ · VS CODE **C3. Abrir el proyecto en VS Code**

File → *Open Folder…* → `C:\dev\heladeria-app` → *Seleccionar carpeta*. Si pregunta si confías en los autores, responde que sí.

Desde ahora usa la **terminal de VS Code** (*Terminal* → *New Terminal*): ya se abre dentro de la carpeta del proyecto.

TÚ · TERMINAL **C4. Primer commit y subida a GitHub**

```
git init
git add .
git commit -m "chore: proyecto inicial con Vite, React y TypeScript"
git remote add origin https://github.com/jj-hesquivel/heladeria-app.git
git push -u origin main
```

La primera vez se abrirá el navegador para que autorices a Git a usar tu cuenta de GitHub (*Git Credential Manager*). Acepta.

> ✓ **Comprueba:** En GitHub, al recargar la página del repositorio, ves los archivos del proyecto.

> **remote / origin.** Un *remote* es una copia del repositorio en otro lugar, aquí GitHub. `origin` es el nombre que se le da por costumbre. `-u` deja recordado que tu rama local `main` se sube a `origin/main`.

TÚ · TERMINAL **C5. Crear la rama develop**

```
git switch -c develop
git push -u origin develop
```

Desde ahora: `main` = lo que está en producción, `develop` = lo terminado que se ensaya, y una rama `feature/…` por cada tarea.

En GitHub: *Settings* → *General* → *Default branch* → cambia a `develop`. Así los Pull Requests apuntan ahí por defecto.

TÚ · EXPLORADOR **C6. Traer el prototipo y las guías al proyecto**

Para que yo tenga el prototipo y estas guías en cualquier computadora, guárdalos dentro del proyecto:

1. Crea la carpeta `C:\dev\heladeria-app\docs\prototipo\`.
2. Copia ahí el `index.html` del prototipo (`Desktop\Mokups\System-ice\index.html`).
3. Copia a `docs\` los archivos de esta guía: `Guia_desarrollo_paso_a_paso.pdf` y `guia-desarrollo.html`, más la guía anterior `Guia_produccion_heladeria.pdf`.

El HTML es la versión que yo puedo leer. El PDF es para ti.

Si prefieres, esto también me lo puedes pedir. Pero el prototipo está en otra carpeta, fuera del proyecto, y tendrás que aprobar el permiso.

6. Etapa D · La base del proyecto

Vamos a dejar el proyecto ordenado y con sus herramientas, todavía sin base de datos. Trabajaremos en una rama nueva.

TÚ · TERMINAL **D1. Rama de trabajo**

```
git switch develop
git switch -c feature/base-proyecto
```

TÚ · TERMINAL **D2. Instalar las dependencias**

```
# las que usa la app
npm install @supabase/supabase-js react-router @tanstack/react-query dexie

# las que solo se usan para programar y probar (-D = devDependencies)
npm install -D supabase prettier vitest jsdom @testing-library/react @testing-library/jest-dom
@testing-library/user-event vite-plugin-pwa husky lint-staged
```

> **dependencies y devDependencies.** Las primeras forman parte de la app que usará la heladería. Las de desarrollo (`-D`) solo sirven para programar, probar y construir, y no llegan al celular. Instalar la CLI de Supabase como dependencia del proyecto hace que las dos computadoras usen **exactamente la misma versión**.

CLAUDE D3. Estructura de carpetas, configuración y CLAUDE.md

Dile a Claude (copia y pega):

```
Lee docs/guia-desarrollo.html y haz el paso D3: crea CLAUDE.md (apéndice B), .gitattributes, .editorconfig, .nvmrc, .env.example, .vscode/extensions.json y .vscode/settings.json; arma la estructura de src/ de la guía; elimina los archivos de ejemplo de Vite y deja App.tsx mínimo. Explícame cada archivo.
```

Lo que haré:

```
heladeria-app/
├─ CLAUDE.md            ← reglas del proyecto para Claude (en cualquier PC)
├─ .gitattributes       ← fin de línea igual en todas las PC
├─ .editorconfig        ← sangría y codificación iguales
├─ .nvmrc               ← versión de Node: 24
├─ .env.example         ← variables necesarias, SIN valores secretos
├─ .vscode/             ← extensiones y ajustes recomendados
├─ docs/                ← guías y prototipo
├─ scripts/             ← tareas de apoyo (crear admin, importar respaldo)
├─ supabase/            ← (se crea en la etapa E)
└─ src/
   ├─ app/              ← App.tsx, rutas, proveedores
   ├─ lib/              ← cliente de Supabase, tipos de la BD, utilidades
   ├─ components/ui/    ← Modal, Hoja deslizante, Botón, Aviso…
   ├─ features/         ← una carpeta por módulo del negocio
   │  ├─ auth/  pedidos/  caja/  personal/  configuracion/
   │  ├─ produccion/  gastos/  sobrantes/  equipos/  panel/
   │  └─ estado-resultados/
   ├─ offline/          ← cola sin conexión (Dexie)
   └─ styles/           ← base.css y temas.css (del prototipo)
```

.gitattributes y fin de línea. Windows termina las líneas de texto con dos caracteres (CRLF) y Linux con uno (LF). Si no se fija una regla, Git marca archivos "cambiados" sin que nadie los haya tocado al pasar de una PC a otra. Este archivo fija LF para todos.

CLAUDE.md. Archivo que leo automáticamente al abrir el proyecto. Tiene las decisiones del proyecto (stack, nombres, reglas de la base de datos, términos como "colaborador"). Gracias a él, en tu casa sigo trabajando igual sin que me repitas todo.

CLAUDE D4. Configurar Prettier, Vitest, Husky y los comandos npm

Dile a Claude (copia y pega):

```
Paso D4 de docs/guia-desarrollo.html: configura Prettier, Vitest (jsdom + Testing Library), Husky con lint-staged (Oxlint y Prettier antes de cada commit) y agrega los scripts npm del apéndice A. Crea una prueba de ejemplo.
```

TÚ · TERMINAL D5. Activar Husky y verificar

```
npm install
npm run lint
npm run format
npm run test:run
npm run build
npm run dev
```

✓ **Comprueba:** Oxlint sin errores, la prueba de ejemplo en verde (*1 passed*), el build termina creando la carpeta `dist/`, y en `localhost:5173` aparece la página mínima.

Husky y lint-staged. **Husky** instala "ganchos" de Git: comandos que corren solos en ciertos momentos. Con **lint-staged**, antes de cada commit se revisan y formatean solo los archivos que vas a guardar. Si algo falla, el commit no se hace. Se activa con `npm install` en cada computadora gracias al script `prepare`.

TÚ · TERMINAL + NAVEGADOR D6. Guardar y unir con un Pull Request

```
git add .
git commit -m "chore: estructura base, herramientas de calidad y pruebas"
git push -u origin feature/base-proyecto
```

En GitHub aparecerá un aviso amarillo "*feature/base-proyecto had recent pushes*" → **Compare & pull request**. Verifica que diga `base: develop ← compare: feature/base-proyecto`, escribe una descripción breve → **Create pull request**. Revisa la pestaña *Files changed* → **Merge pull request** → **Delete branch**.

```
# de vuelta en tu PC: traer develop actualizado y borrar la rama local
git switch develop
git pull
git branch -d feature/base-proyecto
```

Este ciclo (rama → commits → push → Pull Request → merge → pull) es el mismo para **todas** las tareas siguientes.

7. Etapa E · Supabase en tu computadora

TÚ · TERMINAL **E1. Inicializar Supabase**

Docker Desktop debe estar abierto (*Engine running*).

```
git switch -c feature/supabase-local
npx supabase init
```

✓ **Comprueba:** Se crea la carpeta `supabase/` con `config.toml`.

TÚ · TERMINAL **E2. Levantar Supabase local**

```
npm run db:start
```

La **primera vez** descarga varias imágenes de Docker (algunos GB) y puede tardar de 5 a 15 minutos. Las siguientes veces tarda segundos.

> ✓ **Comprueba:** Al final muestra una tabla con las direcciones y llaves. Las importantes:

Servicio	Dirección	Para qué
API (Project URL)	`http://127.0.0.1:54321`	A donde se conecta la app.
Studio	`http://127.0.0.1:54323`	Panel para ver tablas y usuarios.
Mailpit	`http://127.0.0.1:54324`	Buzón falso: aquí llegan los correos que Supabase "envía" en local.
Base de datos	`postgresql://postgres:postgres@127.0.0.1:54322/postgres`	Conexión directa a Postgres.
Publishable key	`sb_publishable_…`	Llave pública que usa la app.
Secret key	`sb_secret_…`	Llave secreta: solo scripts y funciones de servidor.

Para volver a ver esta información: `npm run db:status`.

TÚ · VS CODE E3. Crear tu archivo .env.local

1. En el explorador de VS Code, clic derecho sobre `.env.example` → *Copy*, y luego *Paste* en la misma carpeta.
2. Renombra la copia a `.env.local`.
3. Llénalo con los valores de `npm run db:status`:

```
VITE_SUPABASE_URL=http://127.0.0.1:54321
VITE_SUPABASE_PUBLISHABLE_KEY=sb_publishable_...   ← "Publishable key"
SUPABASE_URL=http://127.0.0.1:54321
SUPABASE_SECRET_KEY=sb_secret_...                  ← "Secret key"
ADMIN_USUARIO=Admin
ADMIN_PASSWORD=escribe-aqui-la-contraseña-de-admin
```

> **Muy importante:** las variables que empiezan con `VITE_` se incrustan en la app y **cualquiera puede verlas** en el celular. Por eso la llave secreta va **sin** `VITE_`: así solo la usan los scripts de tu computadora. `.env.local` está en `.gitignore`, así que **nunca se sube a GitHub**.

Si en tu versión de la CLI las llaves aparecen como *anon key* y *service_role key* (nombres antiguos), usa la *anon* como publishable y la *service_role* como secret.

CLAUDE E4. Configuración de Supabase, cliente y script de administrador

Dile a Claude (copia y pega):

```
Paso E4 de docs/guia-desarrollo.html: en supabase/config.toml pon site_url en
http://localhost:5173, desactiva el registro público y deja la contraseña mínima en 6; crea
src/lib/supabase.ts que lea las variables VITE_ y valide que existan; crea scripts/crear-
admin.mjs que use SUPABASE_URL, SUPABASE_SECRET_KEY, ADMIN_USUARIO y ADMIN_PASSWORD. Explícame
cada cosa.
```

El script de administrador **no tiene la contraseña escrita en el código**: la lee de `.env.local`. El mismo script servirá para crear la cuenta de administración en pruebas y en producción, solo cambiando las variables.

TÚ · TERMINAL E5. Aplicar la configuración y guardar

```
npm run db:stop
npm run db:start
git add .
git commit -m "chore(supabase): entorno local, cliente y script de administrador"
git push -u origin feature/supabase-local
```

Luego haz el Pull Request a `develop`, el merge y el `pull`, igual que en D6.

> ✓ **Comprueba:** Abre Studio (`http://127.0.0.1:54323`). Ves el panel de Supabase con la base vacía.

> **db:stop sin perder datos.** `supabase stop` apaga los contenedores pero **conserva los datos**. Solo los borra si agregas `--no-backup`. Al terminar el día puedes detenerlo para liberar memoria.

> **Pruebas y producción:** En pruebas y producción no hay Docker ni `.env.local`. Los proyectos se crean en supabase.com, sus llaves se configuran en Cloudflare (las `VITE_`) y en GitHub Secrets (la secreta), y la configuración de Auth que hoy está en `config.toml` (registro desactivado, contraseña mínima) se repite en el panel de cada proyecto.

8. Etapa F · Inicio de sesión y roles

Es la primera funcionalidad completa: base de datos, permisos, pantallas y pruebas. Las demás seguirán el mismo camino.

TÚ · TERMINAL **F1. Rama y primera migración**

```
git switch develop
git pull
git switch -c feature/auth-roles
npx supabase migration new perfiles_y_roles
```

✓ **Comprueba:** Se crea un archivo vacío como `supabase/migrations/20261103153000_perfiles_y_roles.sql`. El número es la fecha y hora: así las migraciones se aplican en orden.

CLAUDE **F2. La tabla de perfiles y sus permisos**

Dile a Claude (copia y pega):

```
Paso F2 de docs/guia-desarrollo.html: escribe la migración perfiles_y_roles: tabla perfiles (id = auth.users.id, nombre, usuario único, teléfono opcional, rol admin|colaborador, pago_diario numeric(10,2), activo), función es_admin(), RLS (cada uno ve su perfil, la administración ve y edita todos) y la prueba pgTAP en supabase/tests. Actualiza scripts/crear-admin.mjs para crear también su perfil con rol admin.
```

Usuario sin correo. Supabase Auth inicia sesión con correo. Como las colaboradoras pueden no tener uno, el usuario `erika` se guarda internamente como `erika@heladeria.local`. Ella solo escribe "erika". No se envía ningún correo, porque la confirmación está desactivada.

TÚ · TERMINAL F3. Aplicar, generar tipos, crear la cuenta de administración y probar

```
npm run db:reset
npm run db:types
npm run crear-admin
npm run db:test
```

> ✓ **Comprueba:** `db:reset` termina con *Finished supabase db reset*. Se crea `src/lib/database.types.ts`. `crear-admin` dice que creó a Admin. `db:test` muestra todas las pruebas en *ok*.

En Studio → *Authentication* verás el usuario `admin@heladeria.local`, y en *Table Editor* → *perfiles* su fila con rol `admin`.

> **db:reset.** Borra la base local y la recrea aplicando **todas** las migraciones en orden, más `seed.sql`. Úsalo siempre después de una migración nueva. Ojo: borra los usuarios, así que después vuelve a ejecutar `crear-admin`.

> **db:types.** Lee las tablas y genera los tipos de TypeScript. Así el editor te avisa si escribes mal el nombre de una columna. Repítelo cada vez que cambien las tablas.

CLAUDE F4. Estilos, estructura de la app y pantallas de inicio de sesión

Dile a Claude (copia y pega):

```
Paso F4 de docs/guia-desarrollo.html: lleva los estilos y los 4 temas del prototipo
(docs/prototipo/index.html) a src/styles; arma App con React Router y TanStack Query; crea el
layout con menú ☰ y cabecera; la pantalla de login de colaboradores en / y la de administración
en /admin (sin registro), el manejo de sesión y la protección de rutas por rol. Al entrar:
colaborador → /pedidos, admin → /admin/panel. Agrega pruebas con Testing Library.
```

TÚ · NAVEGADOR F5. Probar a mano

```
npm run dev
```

- Abre `http://localhost:5173/admin` y entra con `Admin` y la contraseña de tu `.env.local`. Debe llevarte al panel.
- Cierra sesión y prueba `http://localhost:5173/` con Admin: debe rechazarlo.
- Prueba una contraseña incorrecta: mensaje de error, sin pistas de qué falló.

CLAUDE F6. Crear cuentas de colaboradores (Edge Function + pantalla Personal)

Dile a Claude (copia y pega):

```
Paso F6 de docs/guia-desarrollo.html: crea la Edge Function crear-usuario (verifica que quien llama es admin y crea el usuario + su perfil con la llave secreta) y la pantalla Personal para crear, editar, desactivar y cambiar la contraseña de colaboradores (nombre, usuario, teléfono opcional, contraseña, pago diario).
```

> **¿Por qué una Edge Function?** Crear usuarios requiere la llave secreta, y esa llave no puede estar en la app. La función corre en el servidor de Supabase, tiene la llave y antes de crear nada comprueba que quien la llama es administradora.

TÚ · TERMINAL + NAVEGADOR F7. Probar y guardar

```
npm run db:stop
npm run db:start
npm run dev
```

Reiniciar Supabase hace que cargue la función nueva. Si la función no responde, ejecuta en **otra** terminal `npx supabase functions serve`.

- Como Admin: *Personal* → *Nuevo colaborador* → crea a `erika`.
- En una **ventana de incógnito** (Ctrl+Shift+N) entra a `localhost:5173` como `erika`: debe ver solo Pedidos, Caja y Configuración.
- Como Erika, escribe a mano `localhost:5173/admin/panel`: debe rechazarla.

```
npm run test:run
npm run db:test
git add .
git commit -m "feat(auth): inicio de sesión, roles y cuentas de colaboradores"
git push -u origin feature/auth-roles
```

Pull Request a `develop` → merge → `git switch develop` → `git pull`.

> **Pruebas y producción:** Las migraciones se aplicarán con `supabase db push` y la Edge Function se publicará con `supabase functions deploy crear-usuario`. La cuenta de administración se creará una vez por entorno con el mismo script, pero con las variables de ese entorno y una **contraseña distinta y fuerte** en producción. En los paneles de Supabase de pruebas y producción también se desactiva el registro público.

9. Etapa G · Los módulos del negocio

Cada módulo sigue **el mismo ciclo de 6 pasos**. Después viene la tabla con el orden y lo que me pides en cada uno.

El ciclo de cada módulo

TÚ · TERMINAL **1. Rama y migración**

```
git switch develop
git pull
git switch -c feature/NOMBRE
npx supabase migration new NOMBRE
```

CLAUDE **2. Pedirme el módulo**

Copia el pedido de la tabla. Yo escribo la migración (tablas, restricciones, índices, RLS), las pruebas de permisos, las funciones de datos, las pantallas y las pruebas de cálculos, llevando la lógica del prototipo.

TÚ · TERMINAL **3. Aplicar y probar**

```
npm run db:reset
npm run db:types
npm run crear-admin
npm run db:test
npm run test:run
npm run dev
```

TÚ · NAVEGADOR **4. Probar a mano como administradora y como colaboradora**

Usa una ventana normal para Admin y una de incógnito para la colaboradora. Compara los resultados con el prototipo: los números deben coincidir.

TÚ · TERMINAL **5. Commit y push**

```
git add .
git commit -m "feat(NOMBRE): descripción corta"
git push -u origin feature/NOMBRE
```

TÚ · NAVEGADOR **6. Pull Request a develop, merge y borrar la rama**

Luego, en la terminal: `git switch develop`, `git pull`, `git branch -d feature/NOMBRE`.

El orden de los módulos

#	Rama / migración	Dile a Claude

G1	`catalogos`	*Paso G1 de docs/guia-desarrollo.html: catálogos (puntos de venta, sabores con base agua/leche y activo, productos con reventa y costo, precios de vaso 5/8/10, empanada 5, gastos fijos) y la pantalla Configuración (tema por celular, datos del negocio solo admin). Con seed.sql de sabores y productos iniciales.*
G2	`cajas`	*Paso G2: cajas (abrir con monto inicial y punto de venta, una abierta por colaborador), reemplazos con entrega de efectivo, y la pantalla Caja para colaborador y admin (todas las cajas abiertas), igual que el prototipo.*
G3	`pedidos`	*Paso G3: pedidos e ítems con sabores (pendiente, pagado efectivo/QR, invitado con para quién). Lo cancelado se borra. Pantalla Pedidos: En curso / Historial, editor de vaso con botones redondos de sabor y precio, igual que el prototipo.*
G4	`pagos-retiros`	*Paso G4: pagos con la caja (hielo, sal, gas, insumos, otros) y retiros, con el panel deslizante del prototipo; cálculo de "debe haber en caja" en la base de datos y cierre de caja con conteo, faltante o sobrante.*
G5	`resumen-pdf`	*Paso G5: lleva el generador de PDF del prototipo (hacerPDF, resumenPDF) a TypeScript, con pruebas, y el botón Resumen PDF en la caja y al cerrarla.*
G6	`produccion-gastos`	*Paso G6: producción por lotes con insumos y costo por litro; compras y gastos; jornadas del personal. Solo admin.*
G7	`sobrantes-equipos`	*Paso G7: sobrantes guardados y recuperados con merma valorizada; equipos con depreciación, roturas y reparaciones. Solo admin.*
G8	`estado-resultados`	*Paso G8: estado de resultados como vista o función SQL (como calcER del prototipo, con mermas, invitaciones, fijos prorrateados y depreciación) y el Panel. Pruebas que comparen con números calculados a mano.*
G9	`importar-respaldo`	*Paso G9: scripts/importar-respaldo.mjs que lea el JSON de respaldo del prototipo y lo cargue en las tablas. Que se pueda ejecutar varias veces sin duplicar.*

Regla de las migraciones: mientras una migración solo existe en tu rama, puedes corregirla y volver a hacer `db:reset`. Cuando ya se aplicó en pruebas o producción, **nunca se edita**: se crea una migración nueva que haga el cambio.

Pruebas y producción: Cada merge a `develop` aplicará las migraciones nuevas al Supabase de pruebas y publicará la app de pruebas automáticamente. Lo configuraremos en la parte 2.

10. Etapa H · Funcionar sin internet (PWA)

CLAUDE **H1. Instalable y con cola sin conexión**

Dile a Claude (copia y pega):

```
Paso H1 de docs/guia-desarrollo.html: configura vite-plugin-pwa (manifest en español, íconos y service worker con actualización avisada) y la cola sin conexión con Dexie para pedidos, cobros, pagos y retiros: guardar primero en el celular, subir con upsert por id (UUID creado en el celular), reintentar al volver la señal y mostrar cuántos faltan por subir. Con pruebas.
```

Para los íconos necesito una imagen cuadrada de al menos 512×512 px (el logo o un helado). Si no tienes, te genero uno simple.

TÚ · NAVEGADOR **H2. Probar el modo sin conexión**

```
npm run build
npm run preview
```

El service worker solo funciona con el build, no con `npm run dev`. Abre la dirección que muestra `preview` (`localhost:4173`):

1. Inicia sesión como colaboradora y abre una caja.
2. Presiona F12 → pestaña *Network* → cambia *No throttling* por **Offline**.
3. Crea y cobra dos pedidos. Debe aparecer el aviso de "2 por subir".
4. Vuelve a *No throttling*. El aviso desaparece y en Studio aparecen los pedidos, sin duplicados.

Para probar en tu celular dentro de la misma red Wi-Fi, ejecuta `npm run dev -- --host` y abre la IP que muestra. La instalación como app y el modo sin conexión completo solo funcionan con HTTPS, así que en el celular se prueban en el entorno de pruebas (parte 2).

11. Etapa I · Pruebas automáticas

Ya vienes escribiendo pruebas en cada módulo. Aquí se agregan las de extremo a extremo.

TÚ · TERMINAL **I1. Instalar Playwright**

```
git switch develop
git pull
git switch -c feature/pruebas-e2e
npm init playwright@latest
```

Respuestas: TypeScript → **sí**; carpeta → `e2e`; GitHub Actions workflow → **no** (lo haremos en la parte 2); instalar navegadores → **sí**.

CLAUDE **I2. Escribir los flujos críticos**

Dile a Claude (copia y pega):

```
Paso I2 de docs/guia-desarrollo.html: configura Playwright con pantalla de celular (360x800) y escribe estas pruebas: login de admin y de colaboradora (y rechazo cruzado), abrir caja, crear y cobrar pedido, cancelar pedido (se borra), pago y retiro, cerrar caja con faltante y descargar el PDF; y el flujo sin conexión.
```

TÚ · TERMINAL **I3. Ejecutar todo**

```
npm run db:reset
npm run crear-admin
npm run test:run
npm run db:test
npm run e2e
```

✓ **Comprueba:** Todo en verde. Si algo falla, `npx playwright show-report` abre un informe con capturas de pantalla del momento del error.

Pruebas y producción: En la parte 2, GitHub Actions ejecutará estas mismas pruebas automáticamente en cada Pull Request, con un Supabase local dentro de GitHub. Si alguna falla, no se debería unir el cambio.

12. Trabajar desde dos computadoras y con otras personas

12.1 Preparar la computadora de tu casa (una sola vez)

TÚ · TERMINAL **Clonar y dejar listo**

Después de hacer la etapa A completa en esa computadora:

```
cd C:\dev
git clone https://github.com/jj-hesquivel/heladeria-app.git
cd heladeria-app
git switch develop
npm install
npm run db:start
```

Luego crea `.env.local` igual que en E3, con los valores de `npm run db:status` de **esa** computadora, y ejecuta:

```
npm run db:reset
npm run crear-admin
npm run dev
```

Abre la carpeta en VS Code; acepta instalar las extensiones recomendadas y abre Claude Code. Como existe `CLAUDE.md`, ya conozco el proyecto. Dime: "*Seguimos con el paso X de docs/guia-desarrollo.html*".

¿Y los datos de mi base local? No viajan entre computadoras, y está bien: la base local es de práctica. La **estructura** sí viaja (las migraciones están en Git) y los datos iniciales vienen de `seed.sql`. Con `db:reset` las dos computadoras quedan iguales.

12.2 Rutina de cada sesión de trabajo

Al empezar	Al terminar (aunque no hayas acabado)
1. Abrir Docker Desktop. 2. `git switch develop` y `git pull` (o `git switch feature/X` y `git pull` si sigues una tarea). 3. Si cambió `package-lock.json`: `npm install`. 4. `npm run db:start`. 5. Si llegaron migraciones nuevas: `npm run db:reset`, `npm run crear-admin` y `npm run db:types`. 6. `npm run dev`.	1. `git add .` 2. `git commit -m "wip: lo que estaba haciendo"` 3. `git push` (la primera vez en la rama: `git push -u origin feature/X`). 4. Opcional: `npm run db:stop` y cerrar Docker. **Nunca dejes trabajo sin subir** al cambiar de computadora: lo que no está en GitHub no existe en la otra.

wip. Significa *work in progress*, trabajo en curso. Es válido hacer commits "wip" en tu rama: lo importante es que el trabajo viaje.

12.3 Si aparece un conflicto

Pasa si editaste el mismo archivo en las dos computadoras sin hacer `pull` antes. Git marca el archivo con `<<<<<<<`, `=======` y `>>>>>>>`. VS Code muestra botones: *Accept Current*, *Accept Incoming*, *Accept Both*. Si no

sabes cuál, pídeme: "*Tengo un conflicto en ARCHIVO, ayúdame a resolverlo*". Después: `git add .`, `git commit` y `git push`.

12.4 Si se suman otras personas

- En GitHub: *Settings → Collaborators → Add people*. Cada persona hace la etapa A y el 12.1.
- Cada uno trabaja en su propia rama `feature/…` y nadie sube directo a `develop` ni a `main`.
- Todo cambio entra por Pull Request y, idealmente, otra persona lo revisa antes del merge.
- Las reglas comunes viven en `CLAUDE.md` y `README.md`. Si cambia una decisión, se actualizan ahí.
- Las llaves secretas de pruebas y producción **no se comparten por chat**: se cargan una sola vez en Cloudflare y GitHub Secrets.

13. Lo que viene: pruebas y producción

Cuando termines esta guía tendrás el sistema completo funcionando en tu computadora. Así se verán las siguientes partes; ninguna requiere cambiar el código.

Tarea	Parte 2 · Pruebas	Parte 3 · Producción
Cuenta	Crear cuenta en supabase.com y cloudflare.com, con 2FA	Las mismas
Base de datos	Proyecto `heladeria-pruebas`, región São Paulo	Proyecto `heladeria-prod`, región São Paulo
Migraciones	`supabase link` + `supabase db push`, automático al unir a `develop`	Igual, al unir a `main`
Edge Functions	`supabase functions deploy`	Igual
App	Cloudflare Pages, rama `develop`	Cloudflare Pages, rama `main`
Variables	`VITE_SUPABASE_URL` y `VITE_SUPABASE_PUBLISHABLE_KEY` en Cloudflare (Preview)	Las mismas en Cloudflare (Production)
Secretos	GitHub Secrets: token, contraseña de la BD, id del proyecto	Los de producción, aparte
Admin	`crear-admin` con variables de pruebas	`crear-admin` con contraseña fuerte
Auth	Desactivar registro y confirmación de correo en el panel	Igual
CI	GitHub Actions corre todas las pruebas en cada PR	Igual, y no se une a `main` sin que pasen
Respaldos	No hace falta	Respaldo nocturno con GitHub Actions y simulacro de restauración
Monitoreo	Opcional	Sentry y UptimeRobot

Cuando quieras pagar (Supabase Pro, dominio propio): se cambia el plan en el panel de facturación o se conecta el dominio en Cloudflare. El código, las migraciones y los procesos siguen exactamente iguales.

14. Problemas comunes y soluciones

Lo que ves	Qué hacer
"*la ejecución de scripts está deshabilitada en este sistema*"	Paso A4: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.
"*No se reconoce el término 'node'*" (o git, docker)	Cierra y abre de nuevo VS Code o PowerShell después de instalar. Si sigue, reinicia la computadora.
`supabase start`: "*Cannot connect to the Docker daemon*"	Docker Desktop no está abierto. Ábrelo y espera a que diga *Engine running*.
"*port 54321 (o 5432x) is already allocated*"	Hay otro Supabase encendido. Ejecuta `npx supabase stop --all` y vuelve a `npm run db:start`.
"*Failed to start inspector … port 9239 in use*"	Una variable de depuración quedó activa en la terminal. Ejecuta `Remove-Item Env:BUN_INSPECT` y repite, o abre una terminal nueva.
`Port 5173 is in use`	Ya tienes otro `npm run dev` abierto en otra terminal. Ciérralo con Ctrl+C o usa la dirección nueva que te muestra.
La app dice que faltan variables de Supabase	Falta `.env.local` o tiene mal un nombre. Compáralo con `.env.example` y reinicia `npm run dev`: Vite solo lee el `.env` al arrancar.
Después de `db:reset` no puedo entrar	El reset borra los usuarios. Ejecuta `npm run crear-admin`.
El editor marca errores en columnas que sí existen	`npm run db:types` y, si persiste, *Ctrl+Shift+P → TypeScript: Restart TS Server*.
`git push` rechazado: "*Updates were rejected… fetch first*"	Hay cambios en GitHub que no tienes (quizá de la otra PC). `git pull`, resuelve conflictos si hay y vuelve a hacer `git push`.
El commit falla por Husky	Lee el mensaje: es Oxlint o Prettier. `npm run format` y `npm run lint`, corrige y repite el commit. No uses `--no-verify`.
Muchos archivos "cambiados" que no tocaste	Fin de línea distinto. Verifica que exista `.gitattributes` y pídeme ayuda antes de hacer commit.
Cualquier otro error	Copia el **mensaje completo** de la terminal y pégamelo, diciendo en qué paso estabas.

15. Apéndice A · Comandos del proyecto

Todos se ejecutan con `npm run NOMBRE` desde la carpeta del proyecto. Los creo en el paso D4. Usar estos nombres en lugar de los comandos largos hace que funcionen igual en cualquier computadora.

Comando	Qué hace por dentro	Cuándo usarlo
`dev`	`vite`	Programar: app en `localhost:5173`.
`build`	`tsc -b && vite build`	Comprobar que compila; genera `dist/`.
`preview`	`vite preview`	Ver el build (para PWA y sin conexión).
`lint`	`oxlint`	Revisar el código.
`format`	`prettier --write .`	Ordenar el formato.
`test` / `test:run`	`vitest` / `vitest run`	Pruebas (modo vigilante / una vez).
`e2e`	`playwright test`	Pruebas de extremo a extremo.
`db:start`	`supabase start`	Encender Supabase local.
`db:stop`	`supabase stop`	Apagarlo (conserva datos).
`db:status`	`supabase status`	Ver direcciones y llaves.
`db:reset`	`supabase db reset`	Recrear la base desde las migraciones.
`db:types`	`supabase gen types > src/lib/database.types.ts`	Después de cambiar tablas.
`db:test`	`supabase test db`	Pruebas de permisos (pgTAP).
`crear-admin`	`node --env-file=.env.local scripts/crear-admin.mjs`	Después de cada `db:reset`.

Puertos que usa el proyecto

Puerto	Qué es
5173	App en desarrollo (Vite)
4173	App construida (`preview`)
54321	API de Supabase
54322	PostgreSQL
54323	Supabase Studio
54324	Mailpit (correos de prueba)

Git de bolsillo

Comando	Qué hace
`git status`	Qué archivos cambiaron y en qué rama estás.

`git switch develop`	Cambiar de rama.
`git switch -c feature/x`	Crear una rama nueva y pasarte a ella.
`git pull`	Traer cambios de GitHub.
`git add .` + `git commit -m "…"`	Guardar cambios.
`git push`	Subir a GitHub.
`git log --oneline -10`	Últimos 10 commits.
`git restore ARCHIVO`	Descartar cambios no guardados de un archivo (con cuidado: no se recuperan).

16. Apéndice B · Contenido de CLAUDE.md

Esto es lo que crearé en el paso D3. Si una decisión cambia, se actualiza aquí y en el archivo.

```
# Heladería App

Sistema interno (no para clientes) de una heladería artesanal en Bolivia (moneda: Bs).
Usuarios: administración (la dueña) y colaboradores. Interfaz y mensajes en español.
Usar siempre el término "colaborador", nunca "trabajadora".

## Stack
React 19 + TypeScript + Vite · React Router · TanStack Query · Supabase (Postgres, Auth, RLS,
Edge Functions) · Dexie (cola sin conexión) · vite-plugin-pwa · Vitest + Testing Library ·
pgTAP · Playwright · Oxlint + Prettier + Husky.

## Entornos
desarrollo = Supabase local (Docker) + .env.local · pruebas = rama develop · producción = rama main.
El código es el mismo en todos; solo cambian las variables de entorno.

## Reglas de la base de datos
- Todo cambio de estructura es una migración nueva en supabase/migrations. Nunca editar una
  migración ya aplicada en pruebas o producción.
- RLS activado en TODAS las tablas, con pruebas pgTAP en supabase/tests.
- Dinero: numeric(10,2). Fechas: timestamptz con default now(). Ids: uuid.
- Nombres de tablas y columnas en español, snake_case.
- Pedidos cancelados se BORRAN (no se guardan).
- Usuarios sin correo: usuario@heladeria.local. Sin registro público; las cuentas las crea la
  administración vía la Edge Function crear-usuario.

## Reglas del frontend
- Estructura por funcionalidad en src/features/<modulo>.
- Las pantallas no llaman a Supabase directo: usan funciones de datos del módulo.
- Variables VITE_ son públicas. La secret key nunca va en el frontend.
- Precios de vaso 5, 8 y 10 Bs; empanada 5 Bs. Cuatro temas: Clásico, Oscuro, Noche, Fresa y menta.
- Referencia funcional: docs/prototipo/index.html. Guía de pasos: docs/guia-desarrollo.html.

## Forma de trabajo
- El usuario ejecuta los comandos y configura páginas web; Claude escribe el código y lo explica.
- Ramas feature/* → PR a develop → PR a main. Conventional Commits en español.
- Antes de dar algo por terminado: npm run lint, npm run test:run, npm run db:test.
```

Lista de verificación al terminar esta parte

- ☐ Las dos computadoras pueden clonar, levantar y correr el proyecto.
- ☐ Login de administración en `/admin` y de colaboradores en `/`, sin registro.
- ☐ Todos los módulos G1 a G9 funcionando y con números iguales a los del prototipo.
- ☐ Funciona sin conexión y sube lo pendiente sin duplicar.
- ☐ `lint`, `test:run`, `db:test` y `e2e` en verde.
- ☐ Todo unido en `develop` y subido a GitHub.
- ☐ Ningún secreto en Git (revisa que `.env.local` no aparezca en GitHub).